

Credit Risk Model Evaluation: Data Quality and Leakage Assessment

To: Head of Credit Risk **From:** David Mauti

Date: August 17, 2026 **Re:** Mobile Money Fraud Model

1 Data Quality Audit

We ran a full diagnostic pass over the raw mobile money transaction data before any modelling work began, covering missingness rates, duplicate detection, data type validation, and correlation screening. Ten columns showed the most significant problems.

Column	Issue	Evidence	Class	Remediation and Why
<code>txn_id</code>	Exact duplicate rows from retry events or log merge failures.	<code>df.duplicated()</code> found 3,600 exact duplicate rows out of 183,600 (2.0%). Duplicated <code>txn_id</code> count matched exactly.	N/A	Drop duplicates before any split. A duplicate row landing in both train and validation inflates performance estimates by letting the model score rows it has already memorised.
<code>amount</code>	Free-text string, not a number. Three incompatible debit encodings coexist.	<code>dtype</code> is <code>object</code> . Sample values: <code>"1,329/- Dr"</code> , <code>"(1,307/-)"</code> , <code>"-2,136/-"</code> . A direct cast to <code>float</code> yields 100% NaN.	N/A	Apply a regex parser stripping commas, the <code>/-</code> suffix, and brackets, inferring sign from context. Zero parse failures were observed after applying this rule.
<code>txn_time</code>	Three incompatible datetime formats coexist in one column.	Sample: ISO 2026-06-30 14:24, UK 07/04/2026 11:21, long-form Jan 29, 2026 01:17 PM. A single-format parse fails on two thirds of rows.	N/A	<code>pd.to_datetime</code> with <code>errors="coerce"</code> across three sequential formats. Recovers all timestamps with zero residual NaT values.
<code>reg_id</code>	Same customer appears under multiple variants from case and whitespace differences in the KYC field.	Raw <code>reg_id</code> has 11,722 distinct values. After <code>.strip().upper()</code> this collapses to 3,991 unique customers, a 66% reduction.	N/A	Canonicalise to uppercase, whitespace stripped, before any grouping or split. Ensures one customer maps to exactly one group.
<code>agent_id</code>	Missingness concentrated in three regions rather than spread randomly.	Missing 13.69% overall. MAR Arusha 45.2%, Jinja 45.1%, Mwanza 44.8%; all other regions 0.0%. Fraud rate equal between missing and present rows (8.39% vs 8.41%).	MAR	Missingness fully explained by observed <code>region</code> column (a legacy logger issue). A dedicated “Unknown” category is sufficient; the pattern is predictable from existing model inputs.
<code>device_id</code>	Uniform dropout with no detectable driver across any observed column.	Missing 5.91%, flat across all 10 regions (5.6–6.1%). Fraud rate: 8.38% missing vs 8.40% present.	MCAR	Treat missing as “Unknown” category. More complex imputation adds no value and risks introducing artificial structure.
<code>gps_lat</code>	High missingness that no observed column can explain, suggesting dropout depends on the unrecorded location itself.	Missing 36.21% (66,464 rows). Rate uniform across all 10 regions (35.4–36.7%), so <code>region</code> does not explain it. Remote/indoor transactions fail GPS capture.	MNAR	Create a binary <code>gps_missing</code> flag. The absence of a GPS ping may carry predictive signal; imputing coordinates would fabricate geographic data.
<code>gps_lon</code>	Identical dropout pattern to <code>gps_lat</code> .	Missing 36.21%, NaN on exactly the same rows as <code>gps_lat</code> .	MNAR	Handled by the same <code>gps_missing</code> flag as <code>gps_lat</code> .
<code>manual_review</code>	Post-event analyst score stored alongside raw transaction features.	Pearson $ r = 0.981$ with <code>is_fraud</code> . A pre-event predictor on an 8% fraud rate cannot achieve this; the score is assigned only after the transaction is flagged.	N/A	Drop entirely before any feature engineering. Does not exist at real-time scoring time. See Section 3.
<code>settlement_status</code>	Categorical outcome written after the transaction lifecycle completes.	Fraud rate by status: <code>reversed</code> = 100.0%, <code>held</code> = 65.1%, <code>pending</code> = 0.0%, <code>settled</code> = 0.4%. Settlement resolves hours to days after the fact.	N/A	Drop entirely. No settlement outcome is observable when a transaction is submitted for scoring. See Section 3.

2 Feature Dictionary

We engineered 15 features spanning five families. All customer-level aggregates are computed strictly over transactions timestamped before `SCORING_TS` = 2026-08-01, ensuring no future information bleeds into feature values.

Feature	Definition	Window	Upstream Fields	Write Time	Fairness Note
amount_abs	Absolute value of the transaction amount.	Single txn	amount	Ledger write at transaction initiation.	–
log_amt	$\log_{1p}(\text{amount_abs})$. Compresses the high-value right tail.	Single txn	amount	Same as above.	–
hr_sin	Sine encoding of transaction hour (0-23), preserving the cyclic midnight relationship.	Single txn	txn_time	Timestamp recorded at transaction initiation.	–
hr_cos	Cosine encoding of transaction hour; paired with hr_sin.	Single txn	txn_time	Same as above.	–
dow_sin	Sine encoding of day of week (0=Mon, 6=Sun).	Single txn	txn_time	Same as above.	–
dow_cos	Cosine encoding of day of week; paired with dow_sin.	Single txn	txn_time	Same as above.	–
payday_dist	Days to nearest payday (1st or 28th). Captures salary-cycle patterns.	Single txn	txn_time	Same as above.	–
gps_missing	Binary flag: 1 if GPS coordinates are null.	Single txn	gps_lat, gps_lon	Set at transaction time when GPS fix fails.	Rural customers may show higher rates due to infrastructure gaps; monitor for geographic disparate impact.
recency_days	Days since the customer's last transaction before SCORING_TS.	All history < SCORING_TS	txn_time, customer_id	txn_time at initiation; customer_id from KYC reg_id at account opening.	–
frequency	Total transaction count before SCORING_TS.	All history < SCORING_TS	txn_time, customer_id	Same as above.	–
monetary_out	Total outgoing (debit) value before SCORING_TS.	All history < SCORING_TS	amount, customer_id	amount written to ledger at initiation.	–
monetary_in	Total incoming (credit) value before SCORING_TS.	All history < SCORING_TS	amount, customer_id	Same as above.	–
n_counterparty	Count of distinct counterparties. High values may indicate smurfing.	All history < SCORING_TS	counterparty, customer_id	counterparty recorded at transaction creation.	–
cashout_ratio	Fraction of transactions that are cash withdrawals.	All history < SCORING_TS	txn_type, customer_id	txn_type set at transaction creation.	–
out_in_ratio	$\text{monetary_out} / (\text{monetary_in} + 1)$. Flags accounts draining faster than they receive.	All history < SCORING_TS	amount, customer_id	Same as monetary_out.	–

3 Leakage Assessment Memo

Two columns in the raw dataset carry information that does not exist at the time a real-time fraud decision must be made. Left in the training set, they produce offline metrics that dramatically overstate how well the model will actually perform.

Background: the six causes of data leakage (Section 2.7.2). The course material identifies six mechanisms by which future or outcome-related information contaminates model training: (1) including the target variable directly as a feature; (2) fitting preprocessing steps on the full dataset before splitting; (3) including post-event fields written only after the outcome is known; (4) allowing duplicate records to straddle the train and validation split; (5) failing to group related records (e.g., same customer) on the same side of the split; and (6) using proxy variables that encode the label without naming it directly. Both leaks identified below fall under causes 3 and 6.

Leak 1: manual_review_score

Where it occurs. This numeric column is stored directly in the transaction ledger alongside raw event data, making it easy to accidentally include in a feature matrix.

How the leakage happens. The score is generated by a fraud analyst who reviews a transaction only after it has already been flagged as suspicious. It encodes the analyst's conclusion about whether fraud occurred (Cause 3: post-event field) and functions as a near-perfect proxy for the target label (Cause 6: proxy variable). Both causes apply simultaneously.

Impact. Including this column produces a cross-validated ROC-AUC of **1.0000**. Removing it brings the score down to **0.6252**, a reduction of **0.3748 AUC points**. Any model selected on the inflated metric would have been chosen entirely on the basis of this one column.

Governance recommendation. The fraud review score should be stored in a separate operational table with a strict write timestamp, never joined to the raw transaction ledger used for model development. All feature store queries must enforce a point-in-time cutoff and explicitly exclude tables written after the scoring timestamp.

Leak 2: `settlement_status`

Where it occurs. This categorical field sits in the same transaction table as the raw input features and describes the final resolution of the payment.

How the leakage happens. Settlement is a downstream business process that resolves hours or days after the transaction is submitted. The value “reversed” is only written once a fraud dispute has concluded (Cause 3: post-event field). Because reversed transactions correspond to confirmed fraud at a rate of 100%, the field is also a near-perfect class separator (Cause 6: proxy variable). At real-time scoring, the field would show only “pending” for every new transaction, providing no information.

Impact. Even after removing `manual_review_score`, retaining `settlement_status` sustains a near-perfect AUC because the “reversed” category creates a 100% fraud-rate split on its own. The combined inflation from both columns together is **0.3748 AUC points** (from 0.6252 to 1.0000). The joint effect confirms that both columns must be excluded simultaneously.

Governance recommendation. Settlement status should be treated as an outcome field, not an input field. It should reside in a separate post-settlement table that is excluded from all feature pipelines by default. Access to outcome tables should require a documented exception reviewed by the model risk team.

4 Reproducible Machine Learning Pipeline

The pipeline is implemented as a single scikit-learn `Pipeline` object. All feature engineering, imputation, scaling, and encoding steps run inside the pipeline, which means they are re-fitted from scratch on each training fold during cross-validation. This eliminates preprocessing leakage by construction.

Architecture. A custom `FeatureEngineer` transformer handles amount parsing, datetime extraction, cyclical encoding, missingness flagging, and customer-level RFM aggregation. It is followed by a `ColumnTransformer` that applies `SimpleImputer` and `RobustScaler` to numeric features and `SimpleImputer` plus `OneHotEncoder` to categorical features. A logistic regression classifier sits at the end of the chain.

Cross-validation. `GroupKFold` with 5 splits groups records by the canonicalised `customer_id`. This ensures no customer’s transactions appear on both sides of a fold boundary, preventing Leakage Cause 5.

Result. The honest, leakage-free cross-validated ROC-AUC is **0.6037 (± 0.0073)**. For comparison, the contaminated model (both leaky columns included) scored 1.0000, confirming the 0.3748 inflation documented in Section 3.

Serialisation. The trained pipeline is saved to `pipeline.joblib` using `joblib.dump`. It can be reloaded with `joblib.load("pipeline.joblib")` and applied to a new raw transaction table without any preprocessing steps outside the object.